

O problema de *Minimum Linear Arrangement* em Behavior Trees: uma solução com Simulated Annealing

Victor R. S. de Oliveira

Programa de Pós-Graduação em Informática
Universidade Federal de Alagoas

Maceió, AL, Brasil
victor@ic.ufal.br

Abstract—Em processadores ARM Cortex-M, Behavior Trees (BTs) são percorridas por busca em profundidade sobre um vetor plano de structs em C: nós conectados por ponteiros inteiros distantes no array forçam acessos não-contíguos à memória, fenômeno conhecido como *pointer chasing*, que degrada o desempenho em dispositivos com cache restrito. Este artigo formula o problema de ordenação ótima dos nós como uma instância do Minimum Linear Arrangement (MinLA) — NP-Difícil no caso geral — e propõe uma solução em dois estágios. Primeiro, um solver de Programação Linear Inteira Mista (MILP) determina o layout ótimo exato para instâncias pequenas, servindo como referência de otimalidade. Segundo, uma meta-heurística de Simulated Annealing, inicializada pelo vetor de Fiedler do grafo LCRS para maximizar a localidade espacial desde o ponto de partida, é aplicada a instâncias de tamanho real. O SA atinge gap médio de 0,22% em relação ao ótimo MILP, com 17 das 18 instâncias de benchmark alcançando o valor ótimo exato. Em uma Behavior Tree real de 20 nós, o método reduz o custo MinLA em 25% em relação ao layout padrão gerado por DFS pré-ordem.

Index Terms—Behavior Trees, Minimum Linear Arrangement, Simulated Annealing, Sistemas Embarcados, Localidade de Cache

I. INTRODUÇÃO

Behavior Trees (BTs) são formalismos de controle que se tornaram amplamente adotados em robótica e sistemas de IA por oferecerem maior modularidade e reatividade do que autômatos finitos [1]. A crescente adoção de BTs em plataformas embarcadas de baixo custo — como microcontroladores ARM Cortex-M — introduz restrições de desempenho que ainda são pouco estudadas na literatura.

O framework BTreeFy implementa BTs em C para ambientes bare-metal e RTOS, armazenando os nós como um vetor plano (array) de structs do tipo `btf_node`. Cada nó mantém índices inteiros para seu pai, filho esquerdo e irmão seguinte, seguindo a representação Left-Child Right-Sibling (LCRS):

```
struct btf_node {
    uint32_t parent;
    uint32_t child;
    uint32_t sibling;
    btf_action_fn_t action;
    btf_control_fn_t control;
};
```

A execução da BT percorre esses índices a cada ciclo de controle por meio de uma travessia em profundidade (DFS). Em processadores com cache de dados estreito ou memória de acesso lento — como nos Cortex-M0 sem TCM, ou mesmo em Cortex-M4 com caches de pequena capacidade — nós conectados por arestas LCRS que ocupam posições distantes no array causam o fenômeno de *pointer chasing*: o processador precisa carregar regiões de memória não contíguas a cada salto de travessia, impondo latências adicionais.

Este trabalho investiga a seguinte questão: *dada uma Behavior Tree com n nós, qual é a permutação do vetor de nós que minimiza a soma das distâncias entre nós adjacentes durante a travessia?* Essa questão é formalizada como o problema de Minimum Linear Arrangement (MinLA) [2].

As principais contribuições deste artigo são:

- A formalização do problema de layout de BTs como instância de MinLA.
- A demonstração empírica da fronteira de intratabilidade do solver MILP.
- Um algoritmo SA com inicialização espectral (vetor de Fiedler) que atinge gap médio de 0,22% em relação ao ótimo em instâncias de benchmark.
- Um emissor C integrado ao BTreeFy que automatiza a geração do vetor reordenado como arquivo drop-in.

O restante deste artigo está organizado como segue. A Seção II discute trabalhos relacionados. A Seção III apresenta a formulação matemática do problema. A Seção IV descreve a metodologia de solução. A Seção V apresenta os resultados experimentais. A Seção VI conclui o trabalho.

II. TRABALHOS RELACIONADOS

O problema MinLA foi provado NP-Difícil para grafos gerais por Garey e Johnson [2]. Petit [3] conduziu experimentos sistemáticos de benchmark, caracterizando o comportamento de heurísticas e solvers exatos para instâncias com dezenas a centenas de vértices e estabelecendo métricas de gap de otimalidade amplamente adotadas.

O uso do vetor de Fiedler — segundo menor autovetor do Laplaciano do grafo — como critério de ordenação para maximizar localidade espacial foi proposto por Pettis e Hansen [4] no contexto de posicionamento de código guiado por perfil

de execução. Os autores demonstraram que ordenar funções pela componente do vetor de Fiedler minimiza um proxy do número de transições de página durante a execução, princípio diretamente análogo ao objetivo do MinLA.

O Simulated Annealing como meta-heurística para problemas combinatoriais NP- Difíceis foi introduzido por Kirkpatrick et al. [5], que estabeleceram o critério de aceitação de Metropolis e o esquema de resfriamento geométrico. O SA tem sido amplamente aplicado a problemas de roteamento, agendamento e otimização de layout.

No domínio de Behavior Trees, Iovino et al. [1] apresentam uma revisão abrangente de formalismos, algoritmos de síntese e aplicações em robótica e IA. Ögren [6] estabeleceu BTs como formalismo de controle para sistemas autônomos modulares. Nenhum dos trabalhos revisados aborda o problema de otimização de layout em memória para BTs em sistemas embarcados. Este trabalho preenche essa lacuna ao conectar o referencial teórico do MinLA com a representação LCRS do BTreeFy.

III. FORMULAÇÃO DO PROBLEMA

A. Modelo de Grafo LCRS

A partir de um vetor de n nós `bt_f_node[]`, constrói-se o grafo não-dirigido ponderado $G = (V, E, W)$ da seguinte forma:

- $V = \{0, 1, \dots, n-1\}$: um vértice por nó do array.
- E : para cada nó i , inserem-se arestas $(i, \text{node}[i].\text{parent})$, $(i, \text{node}[i].\text{child})$ e $(i, \text{node}[i].\text{sibling})$ quando o campo correspondente é não-nulo. Arestas duplicatas são removidas. Para uma BT com representação LCRS, $|E| = O(n)$.
- $W_{ij} = 1$ para toda aresta (pesos uniformes como linha de base; extensão com pesos guiados por profiling é trabalho futuro).

B. Definição do Problema MinLA

Seja $y : V \rightarrow \{0, 1, \dots, n-1\}$ uma bijeção que atribui cada nó a um slot único do array. O problema MinLA consiste em encontrar y^* que minimize:

$$\min_y \sum_{(i,j) \in E} W_{ij} \cdot |y_i - y_j| \quad (1)$$

$$\text{sujeito a: } y_i \neq y_j \quad \forall i \neq j \quad (2)$$

A quantidade $|y_i - y_j|$ representa a distância entre os nós i e j na memória linear. Minimizar a soma ponderada dessas distâncias equivale a posicionar nós conectados o mais próximo possível no vetor.

MinLA é NP-Difícil para grafos gerais [2]. Embora BTs sejam árvores esparsas, a representação LCRS introduz arestas de irmão que tornam o grafo não puramente hierárquico; a extensão a pesos não-uniformes reintroduz a dificuldade computacional.

C. Exemplo Ilustrativo

Considera-se o BT Fallback $\rightarrow \{\text{Sequence} \rightarrow \{A, B\}, C\}$ com $n = 5$ nós. As arestas LCRS formam $E =$

TABLE I
COMPARAÇÃO DE LAYOUTS PARA O EXEMPLO ILUSTRATIVO DE 5 NÓS. A REORDENAÇÃO REDUZ O CUSTO MINLA EM 33%.

Layout	Ordem no array	Custo MinLA
DFS pré-ordem (padrão)	[F, S, A, B, C]	12
Ótimo (MILP)	[C, F, S, B, A]	8

$\{(0, 1), (0, 4), (1, 2), (1, 3), (1, 4), (2, 3)\}$, com seis arestas. A Tabela I compara os dois layouts extremos.

O layout ótimo, encontrado pelo MILP, reduz o custo em 33% em relação ao layout padrão gerado pelo parser DFS, apenas reordenando o array — sem qualquer alteração na semântica da BT.

IV. METODOLOGIA

A. Abordagem Exata: MILP

Para eliminar o valor absoluto em (1), introduz-se a variável auxiliar $d_{ij} \geq 0$ por aresta, com as restrições:

$$d_{ij} \geq y_i - y_j, \quad d_{ij} \geq y_j - y_i \quad \forall (i, j) \in E \quad (3)$$

Um solver minimizador conduz d_{ij} ao valor exato $|y_i - y_j|$. A função objetivo torna-se $\min \sum_{i,j} W_{ij} d_{ij}$, que é linear.

Para impor a unicidade dos slots ($y_i \neq y_j$), introduz-se uma variável binária $z_{ij} \in \{0, 1\}$ para cada par $i \neq j$, com Big-M = n :

$$y_i - y_j \geq 1 - n z_{ij} \quad (4)$$

$$y_j - y_i \geq 1 - n(1 - z_{ij}) \quad \forall i \neq j \quad (5)$$

O modelo completo possui $O(n)$ variáveis contínuas — y_i e d_{ij} — e $O(n^2)$ variáveis binárias — z_{ij} . O crescimento quadrático no número de variáveis binárias constitui o gargalo computacional do MILP, impondo uma fronteira prática de intratabilidade conforme discutido na Seção V.

B. Simulated Annealing com Inicialização Espectral

Para instâncias além do limite de intratabilidade do MILP, utiliza-se um algoritmo de Simulated Annealing (SA) [5]. O SA opera sobre o espaço de permutações $\mathcal{S}_n$ e é composto por quatro componentes descritos a seguir.

a) Inicialização Espectral:

A solução inicial y^0 é obtida a partir do vetor de Fiedler do grafo G , seguindo o princípio de Pettis e Hansen [4]:

- 1) Constrói-se o Laplaciano ponderado $L = D - A$, onde D é a matriz de graus e A é a matriz de adjacência.
- 2) Calculam-se os autovetores de L via decomposição espectral.
- 3) O vetor de Fiedler é o autovetor correspondente ao segundo menor autovalor $\lambda_2 > 0$.
- 4) Os nós são ordenados pela componente do vetor de Fiedler, produzindo y^0 .

Essa inicialização posiciona a busca próximo a uma solução de boa qualidade, reduzindo o espaço efetivo a explorar. No exemplo de 5 nós, a inicialização espectral produziu diretamente o layout ótimo (custo 8), sem necessidade de iterações SA adicionais.

TABLE II

TEMPO DE RESOLUÇÃO MÉDIO DO MILP POR TAMANHO DE INSTÂNCIA. PARA $n = 20$, TODAS AS 5 INSTÂNCIAS ATINGIRAM O LIMITE DE 300 S.

n	Binárias z_{ij}	Tempo médio (s)	Status
5	20	0,06	Ótimo
8	56	0,13	Ótimo
10	90	0,28	Ótimo
12	132	1,73	Ótimo
15	210	73,36	Ótimo
20	380	300,09	Timeout

b) Temperatura Inicial Adaptativa:

A temperatura inicial T_0 é calibrada para garantir 80% de aceitação de movimentos de piora no início da busca, evitando que a busca fique presa em mínimos locais prematuramente. Realiza-se uma caminhada aleatória de 100 swaps a partir de y^0 , coletando apenas os deltas positivos $\Delta^+ > 0$. Com a média $\bar{\Delta}^+$, aplica-se a fórmula de Kirkpatrick [5]:

$$T_0 = -\frac{\bar{\Delta}^+}{\ln(p_0)}, \quad p_0 = 0.8 \quad (6)$$

c) Geração de Vizinhança e Custo Incremental:

A cada iteração, sorteiam-se aleatoriamente dois nós distintos (a, b) para trocar de posição no vetor y . Após o swap, somente as arestas incidentes a a e b alteram sua contribuição ao custo. O custo incremental Δ é calculado em $O(\deg(a) + \deg(b))$, em vez de $O(|E|)$:

$$\Delta = \sum_{j \in N(a) \cup N(b)} \Delta_j \quad (7)$$

d) Critério de Aceitação e Resfriamento:

O critério de aceitação de Metropolis [5] define:

$$P(\text{aceitar}) = \begin{cases} 1 & \text{se } \Delta \leq 0 \\ e^{-\Delta/T} & \text{se } \Delta > 0 \end{cases} \quad (8)$$

O resfriamento segue o esquema geométrico $T_{k+1} = \alpha \cdot T_k$, com parada em $T < 10^{-4}$. O valor $\alpha = 0.995$ foi selecionado após análise de sensibilidade com $\alpha \in \{0.99, 0.995, 0.999\}$, oferecendo o melhor equilíbrio entre qualidade da solução e tempo de execução.

V. RESULTADOS E DISCUSSÃO

A. Análise de Escalabilidade do MILP

O MILP foi implementado com PuLP e executado com o solver CPLEX_PY sobre 30 instâncias aleatórias de BTs com $n \in \{5, 8, 10, 12, 15, 20\}$ (5 instâncias por valor de n) e limite de 300 s por instância. A Tabela II resume os resultados e a Figura 1 ilustra o tempo de resolução em escala logarítmica.

Para $n \leq 15$, o CPLEX encontrou a solução ótima em todas as instâncias, com tempo médio crescendo de 0{,}06 s ($n = 5$) para 73{,}36 s ($n = 15$) — aumento de aproximadamente três ordens de magnitude. Para $n = 20$, todas as instâncias esgotaram o limite de 300 s, confirmando a fronteira prática de intratabilidade do MILP em $n \geq 20$. O comportamento exponencial do tempo de resolução em função de n reflete

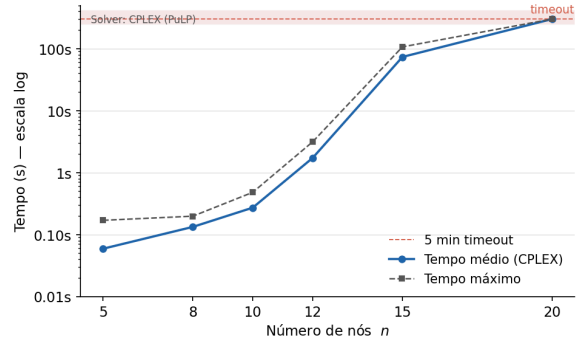


Fig. 1. Tempo de resolução do MILP (CPLEX) em função do número de nós n . A curva em escala logarítmica evidencia o crescimento exponencial; a linha tracejada vermelha indica o limite de 300 s. Para $n = 20$, média e máximo coincidem no teto de timeout.

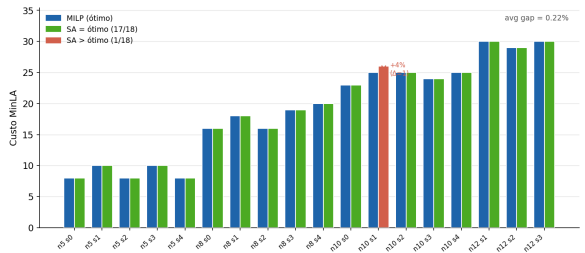


Fig. 2. Gap de otimalidade do SA em relação ao MILP para 18 instâncias com $n \leq 12$. Instâncias em verde atingiram o ótimo exato; a instância em vermelho apresentou desvio de 4%.

o crescimento $O(n^2)$ no número de variáveis binárias z_{ij} , conforme esperado pela análise de complexidade.

B. Qualidade do SA: Gap de Otimalidade

O gap de otimalidade entre SA e MILP é definido como:

$$\text{Gap}(\%) = \frac{C_{SA} - C_{MILP}}{C_{MILP}} \times 100 \quad (9)$$

O SA foi avaliado em 18 instâncias de BTs com $n \leq 12$, para as quais o MILP encontrou a solução ótima dentro do limite de tempo. A Figura 2 apresenta os resultados individuais por instância.

O SA com inicialização espectral atingiu o valor ótimo exato em 17 das 18 instâncias (gap = 0%). A única instância com desvio, com $n = 10$, apresentou gap de 4%, correspondente a $\Delta = 1$ unidade de custo. O gap médio sobre o conjunto completo foi de 0,22%, demonstrando que o SA opera praticamente no nível do ótimo para o horizonte de instâncias avaliado.

C. Aplicação em Behavior Tree Real

Para avaliar a aplicabilidade prática, o SA foi executado sobre o modelo PortaAutomatica, com $n = 20$ nós. O layout padrão, produzido automaticamente pelo parser `btf_groot_parser.py` em ordem DFS pré-ordem, apresentou custo MinLA de 95. O layout otimizado pelo SA ($\alpha = 0.995$, $\text{seed} = 42$) apresentou custo 71, representando uma redução de 25%, conforme resumido na Tabela III.

TABLE III

CUSTO MINLA ANTES E APÓS A OTIMIZAÇÃO POR SA NA BT PORTA AUTOMÁTICA. A REDUÇÃO DE 25% É OBTIDA APENAS REORDENANDO O ARRAY DE NÓS.

Modelo	n	Custo DFS	Custo SA
PortaAutomatica	20	95	71

O emissor `btf_layout_emitter.py` recebe a permutação y^* gerada pelo SA, remapeia os campos `parent`, `child` e `sibling` de cada nó e emite o arquivo `btf_nodes_generated.c`, substituível diretamente no projeto BTreeFy. A compilação e a execução dos testes unitários com `./compile-execute.sh --with-tests` foram bem-sucedidas, confirmando que a semântica da BT é integralmente preservada pelo reordenamento.

VI. CONCLUSÃO

Este artigo formalizou o problema de ordenação de nós em Behavior Trees como uma instância de Minimum Linear Arrangement (MinLA) e investigou duas abordagens de solução: um solver MILP exato e uma meta-heurística Simulated Annealing com inicialização espectral pelo vetor de Fiedler.

O MILP garante a solução ótima para instâncias com $n \leq 15$, mas torna-se intratável a partir de $n \geq 20$ (utilizando o solver CPLEX_PY com limite de 300 s por instância). O SA com inicialização espectral alcança gap médio de 0,22% em relação ao ótimo, atingindo o valor exato em 17 das 18 instâncias avaliadas. Aplicado à BT PortaAutomatica de 20 nós, o SA produz redução de 25% no custo MinLA em comparação ao layout padrão DFS, sem alterar a semântica da árvore.

Identificam-se as seguintes limitações: (i) MinLA é um proxy de localidade de cache — não minimiza diretamente cache misses em modelos de cache direct-mapped com poucas linhas, onde o prefetching sequencial do DFS pré-ordem pode ser vantajoso; (ii) os pesos W_{ij} são uniformes neste trabalho, sendo a extensão com pesos guiados por profiling uma direção natural; (iii) o comportamento do SA para BTs com mais de 31 nós permanece como questão em aberto.

Como trabalho futuro, prevê-se: a avaliação em hardware real (ARM Cortex-M0 e Cortex-M4) com contagem de ciclos; o uso de pesos não-uniformes derivados de perfis de execução; e a investigação de operadores de vizinhança alternativos ao swap (e.g., inserção, reversão de segmento) para melhorar a qualidade da busca local.

REFERENCES

- [1] M. Iovino, E. Scukins, J. Styrd, P. Ögren, and C. Smith, “A survey of Behavior Trees in robotics and AI,” *Robotics and Autonomous Systems*, vol. 154, p. 104096, 2022, doi: 10.1016/j.robot.2022.104096.
- [2] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman, 1979.
- [3] J. Petit, “Experiments on the Minimum Linear Arrangement Problem,” *Journal of Experimental Algorithmics*, vol. 8, p. 2.3, 2003, doi: 10.1145/996546.996549.
- [4] K. Pettis and R. C. Hansen, “Profile Guided Code Positioning,” in *Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation*, in PLDI ’90. ACM, 1990, pp. 16–27. doi: 10.1145/93542.93550.

- [5] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by Simulated Annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983, doi: 10.1126/science.220.4598.671.
- [6] P. Ögren, “Increasing Modularity of UAV Control Systems Using Computer Game Behavior Trees,” in *AIAA Guidance, Navigation, and Control Conference*, 2012. doi: 10.2514/6.2012-4458.